Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

28 de octubre de 2026

Índice

Introducción **3**
1. Objetivos 4
2. Alcance 4
Marco teórico **5**
1. Modelo publicador/suscriptor 6
2. MQTT 8
3. Protocol Buffers 10
   3.1. Esquema y generación de código 10
   3.2. Codificación binaria 11
   3.3. Compatibilidad entre versiones 11
   3.4. Limitaciones 11
4. Stack MQTT y Protocol Buffers 11
5. Comparación con alternativas 11
   5.1. MQTT, HTTP y CoAP 11
   5.2. Protocol Buffers y JSON 11
Desarrollo **12**
1. MQTT sobre TCP y TLS 12
   1.1. Conexión persistente y keep-alive 12
   1.2. Cifrado con TLS 12
   1.3. QoS MQTT sobre la confiabilidad de TCP 12
2. Capa de red 12
   2.1. NAT y firewalls 12
   2.2. Ruteo interno 12
   2.3. Ruteo externo 12
   2.4. Convergencia y conexiones de larga duración 12
3. Ruteo por tópicos y ruteo IP 12
4. Casos de uso 12
5. Diseño experimental 12
   5.1. Plataforma común 12
   5.2. Experimento 1: MQTT contra HTTPS para telemetría 12
   5.3. Experimento 2: sincronización de osciladores acoplados 12
Conclusiones **13**
Bibliografía **14**
Anexos **15**
A. Mensajes del experimento de sincronización 15

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots, implementa la comunicación entre servicios e integraciones desplegadas *on the edge* utilizando MQTT y mensajes definidos con Protocol Buffers, donde brokers desplegados en la nube reciben datos en tiempo real de miles de robots y dispositivos IoT, que a su vez son controlados por el mismo canal, aun en instalaciones con redes inestables. Desarrollado por la misma empresa, el producto de código abierto OpenRobOps libera las definiciones de su protocolo de comunicación en estructuras Protobuf que fijan el formato de las comunicaciones entre robot y plataforma.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

BORRADOR

Marco teórico

Las redes de computadoras existen para hacer posibles las aplicaciones de red, es decir, programas que se ejecutan en distintos sistemas terminales y que se comunican entre sí a través de la red. La lógica de estas aplicaciones reside únicamente en los sistemas terminales, mientras que los nodos intermedios operan en la capa de red e inferiores. En consecuencia, quien desarrolla una aplicación no modifica la red, sino que utiliza los servicios que esta ofrece y define cómo se organiza la comunicación entre los extremos. MQTT y Protocol Buffers se ubican en este plano.

La forma predominante de organizar la comunicación a nivel de aplicación es el modelo cliente-servidor [1]. En este modelo, un servidor permanece activo en una dirección conocida a la espera de solicitudes, y un cliente lo contacta cuando requiere un servicio, envía una solicitud y aguarda la respuesta. La Web constituye el ejemplo característico, en el que el navegador solicita un recurso y el servidor lo devuelve. Se trata de un modelo orientado a la demanda, en el que la información se transmite únicamente cuando es solicitada y el intercambio es iniciado siempre por la parte que la requiere.

Este modelo presenta limitaciones para los sistemas considerados en este trabajo. En ellos, los dispositivos generan datos de manera continua y en instantes que no pueden anticiparse, y deben a su vez recibir comandos originados en la nube. Para que el servidor transmita un comando por iniciativa propia, debería actuar como cliente del dispositivo, lo que requiere que este acepte conexiones entrantes. Esta condición rara vez se cumple cuando el dispositivo se encuentra detrás de un NAT o de un firewall administrado por terceros. La alternativa que ofrece el propio modelo es la consulta periódica por parte del dispositivo, cuyo costo en tráfico y en latencia aumenta con la frecuencia de las consultas.

La capa de transporte ofrece una base para superar esta limitación. TCP establece una conexión persistente *full-duplex*, que permite a ambos extremos transmitir datos en cualquier momento con entrega fiable y ordenada. Si es el dispositivo quien abre la conexión hacia un servidor de dirección conocida, no necesita aceptar conexiones entrantes y conserva el rol de cliente. En ese caso, una vez establecida la conexión, el servidor también puede enviarle mensajes al cliente sin esperar que este realice una solicitud, lo que vuelve innecesaria la consulta periódica.

La conexión TCP, sin embargo, solo garantiza la entrega de una secuencia de bytes entre dos procesos. Qué mensajes se intercambian, con qué formato y en qué momento lo determina el protocolo de la capa de aplicación, que especifica los tipos de mensajes, su sintaxis y su semántica, así como las reglas que indican cuándo y cómo se envía y se responde cada uno de ellos [2].

En los sistemas estudiados, el diseño de ese protocolo está condicionado en primer lugar por la forma en que circula la información. Un mismo dato suele interesar a varios consumidores y cada consumidor recibe datos de muchos dispositivos, por lo que resulta conveniente que productores y consumidores puedan comunicarse sin conocerse entre sí ni estar conectados simultáneamente, que es la propuesta del modelo publicador/suscriptor implementado por MQTT. A esto se suma la representación de los datos dentro de cada mensaje, que debe ser interpretada de manera unívoca por equipos heterogéneos y ocupar poco ancho de banda en enlaces que suelen ser limitados, problema del que se ocupa Protocol Buffers.

BORRADOR

1. Modelo publicador/suscriptor

El modelo publicador/suscriptor (*publish/subscribe*) ofrece precisamente la posibilidad de que productores y consumidores de información se comuniquen sin conocerse entre sí, lo que lo hace adecuado para los sistemas descritos. En este esquema de interacción, los suscriptores expresan su interés en un evento o en un patrón de eventos, y son notificados de forma asíncrona de todo evento generado por un publicador que coincida con ese interés [3]. El modelo básico se apoya en un servicio de notificación de eventos, una abstracción que almacena las suscripciones y entrega cada evento publicado a todos los suscriptores interesados. Las suscripciones permanecen en ese servicio y son invisibles a los publicadores.

De esta manera, una de las características principales del modelo y común a todas sus variaciones es el desacoplamiento total de las entidades que se comunican, que puede descomponerse en tres dimensiones [3]:

- El **desacoplamiento en el espacio** (*space decoupling*) implica que las partes no necesitan conocerse, ya que los publicadores no mantienen referencias a los suscriptores ni saben cuántos participan de la interacción, y lo mismo ocurre en sentido inverso.
- El **desacoplamiento en el tiempo** implica que las partes no necesitan participar de la interacción al mismo tiempo, por lo que un publicador puede publicar eventos mientras un suscriptor está desconectado, y un suscriptor puede recibir la notificación de un evento cuyo publicador ya se desconectó.
- El **desacoplamiento de sincronización** implica que los publicadores no se bloquean al producir eventos y que los suscriptores son notificados mientras realizan otras actividades, fuera del flujo principal de control de ambos.

Este desacoplamiento reduce la coordinación necesaria entre los dispositivos participantes de la comunicación y favorece la escalabilidad en entornos asíncronos por naturaleza.

Estas tres dimensiones permiten contrastar el modelo con el cliente/servidor descrito al comienzo del capítulo. En el caso del paradigma cliente/servidor el cliente debe conocer la dirección del servidor, ambos deben estar activos durante el intercambio y el cliente queda a la espera de la respuesta, por lo que la interacción está acoplada en el espacio, en el tiempo y, del lado del cliente, en la sincronización. La misma comparación puede extenderse a otros paradigmas de comunicación distribuida, como resume la Tabla 1. La invocación remota de procedimientos (RPC) presenta el mismo acoplamiento que el modelo cliente-servidor, mientras que sus variantes asíncronas y el patrón *observer* eliminan únicamente el acoplamiento de sincronización. Los espacios de tuplas y las colas de mensajes desacoplan a productores y consumidores en el espacio y en el tiempo, pero obligan a los consumidores a extraer los datos de forma síncrona. El modelo publicador/suscriptor es el único que reúne las tres propiedades.

BORRA-DOR

Tabla 1: Desacoplamiento que ofrece cada paradigma de comunicación [3, p. 121].

Paradigma	**Espacio**	**Tiempo**	**Sincronización**
Pasaje de mensajes	No	No	Solo el productor
RPC/RMI	No	No	Solo el productor
RPC/RMI asíncrona	No	No	Sí
RPC/RMI con futuros	No	No	Sí
Notificaciones (*observer*)	No	No	Sí
Espacios de tuplas	Sí	Sí	Solo el productor
Colas de mensajes (*pull*)	Sí	Sí	Solo el productor
Publicador/suscriptor	Sí	Sí	Sí

Las variantes del modelo publicador/suscriptor se distinguen por la forma en que los suscriptores especifican los eventos de interés. En el esquema basado en tópicos, el más antiguo, los eventos se publican bajo nombres que los sistemas modernos organizan en jerarquías, y la sintaxis de comodines permite suscribirse a todos los tópicos de la jerarquía que coincidan con el patrón dado. Este esquema puede implementarse de manera muy eficiente, aunque puede resultar limitado. El esquema basado en contenido clasifica los eventos según sus propias propiedades, mediante filtros que define el suscriptor, y ofrece mayor expresividad a costa de protocolos más complejos y de un mayor costo de ejecución. El esquema basado en tipos clasifica los eventos según su tipo, lo que permite verificar en tiempo de compilación que productores y consumidores coinciden en la estructura de los datos. MQTT, descrito en la Sección 2, adopta el esquema basado en tópicos.

La implementación del servicio de eventos admite a su vez distintas arquitecturas. En una arquitectura centralizada, los productores envían los mensajes a una entidad que los almacena y los reenvía a los consumidores. En una arquitectura distribuida no existe tal entidad, y los mecanismos de almacenamiento y reenvío residen en los propios procesos de productores y consumidores, mientras que un enfoque intermedio implementa el servicio como una red de servidores dedicados. En la arquitectura centralizada, que es la que adopta MQTT, la entidad central se denomina *broker*. Este suele comunicarse con publicadores y suscriptores mediante canales punto a punto confiables y conservar copias de los eventos en almacenamiento estable, lo que le permite ofrecer persistencia y entrega garantizada. Como contraparte, el uso de un broker agrega un salto a cada mensaje y concentra la carga de todos los participantes en un punto de falla que tiene el potencial de demorar la entrega de los eventos [3].

Sobre una red TCP/IP, la arquitectura centralizada se apoya en interacciones cliente-servidor. Tanto los publicadores como los suscriptores establecen la conexión hacia el broker, por lo que todos ellos actúan como clientes, y el broker es el único participante que debe tener una dirección conocida y aceptar conexiones entrantes. Su función es la de un sistema intermedio que mantiene conexiones TCP con cada extremo y retransmite los mensajes entre ellas, de manera análoga a los intermediarios definidos para HTTP [4]. La diferencia con el modelo cliente-servidor aparece una vez registrada la suscripción, ya que el broker entrega cada evento por la conexión que el suscriptor estableció previamente, sin requerir de una solicitud. De este modo, los datos llegan al suscriptor en el momento en que se producen, sin que este deba consultar periódicamente ni aceptar conexiones entrantes. Esta capacidad de entregar datos por iniciativa del otro extremo es común a cualquier canal bidireccional persistente, como una conexión WebSocket, pero lo que distingue al modelo publicador/suscriptor es que, sobre esos canales, el broker determina a

BORRADOR

qué suscriptores corresponde cada evento de modo que publicadores y suscriptores permanecen desacoplados aun cuando cada uno mantiene una conexión punto a punto con el broker.

2. MQTT

MQTT es un protocolo de transporte de mensajes de tipo publicador/suscriptor, basado en una arquitectura cliente-servidor y diseñado para ser liviano, abierto y fácil de implementar [5]. La primera versión estandarizada por OASIS fue la 3.1.1, adoptada también como norma ISO/IEC 20922, y la versión 5.0, en la que se centra este trabajo, incorpora numerosas funcionalidades nuevas sin alterar el núcleo del protocolo. MQTT adopta la arquitectura centralizada descrita en la sección anterior denominando *broker* al servidor y cliente a todo programa o dispositivo que abre una conexión hacia él para publicar mensajes y/o suscribirse a ellos, y deja a cada implementación la elección de los mecanismos de autenticación, autorización y cifrado. MQTT delega en la capa de transporte la entrega ordenada y sin pérdidas de los datos a la capa de transporte, por lo que funciona sobre TCP, TLS o WebSocket, pero no directamente sobre UDP.

Cada mensaje de aplicación lleva un nombre de tópico, el cual consiste en una cadena de texto que el servidor compara con las suscripciones de los clientes. Los tópicos se organizan en niveles separados por el carácter `/`, y las suscripciones se expresan mediante filtros que se comparan con los tópicos de mensajes entrantes. Las expresiones de suscripción pueden incluir los comodines `+`, que coincide con exactamente un nivel, y `#`, que solo puede ubicarse al final del filtro y coincide con cualquier cantidad de niveles.

De esta manera las expresiones de suscripción permiten, por ejemplo, en una flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, suscribirse con el filtro `flota/+/bateria` que recibe el estado de batería de todos los robots o el filtro `flota/robot1/#`, que recibe todos los mensajes de el robot `robot1`.

Implementar el esquema basado en tópicos, MQTT implementa desacoplamiento en el espacio, ya que el publicador solo conoce el nombre del tópico y es el servidor quien determina qué clientes reciben cada mensaje. La versión 5.0 agrega además las suscripciones compartidas, en las que cada mensaje se entrega a uno solo de los clientes de un grupo, lo que permite repartir el procesamiento entre varios consumidores.

La entrega de los mensajes se rige por tres niveles de calidad de servicio (QoS), cuyos intercambios de paquetes muestra la Figura 1. Con QoS 0, el mensaje se envía sin confirmación ni reintentos y llega una vez o ninguna. Con QoS 1, el receptor confirma cada mensaje con un paquete PUBACK y el emisor lo conserva hasta recibir esa confirmación, lo que asegura que llegue al menos una vez aunque pueden producirse duplicados. Con QoS 2, un intercambio en dos etapas garantiza que el mensaje llegue exactamente una vez, a costa de una mayor sobrecarga.

El nivel de QoS se aplica por separado entre el publicador y el broker y entre el broker y cada suscriptor, y cada suscriptor recibe el mensaje con el menor nivel de QoS entre el usado al publicarlo y el máximo concedido a su suscripción. La relación entre estos niveles y la confiabilidad que ya ofrece TCP se analiza en la Sección 1.3.

BORRADOR

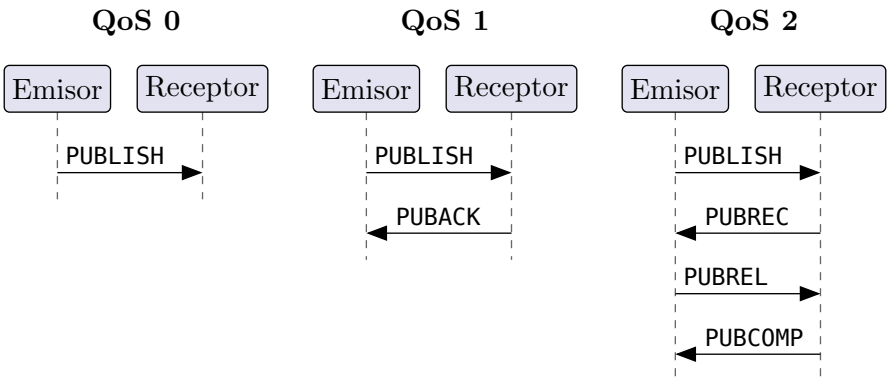


Figura 1: Intercambio de paquetes de control en cada nivel de QoS.

Las confirmaciones de QoS 1 y 2 requieren que cliente y servidor mantengan un estado de sesión. En el servidor, ese estado comprende las suscripciones del cliente y los mensajes destinados a él que aún no fueron enviados o confirmados, y puede conservarse a lo largo de varias conexiones consecutivas. Al conectarse, el cliente indica si inicia una sesión nueva o retoma la anterior, y durante cuánto tiempo debe conservarse el estado una vez cerrada la conexión. Este mecanismo es el que permite el desacoplamiento en el tiempo, ya que el servidor almacena los mensajes dirigidos a un suscriptor desconectado y se los entrega cuando este vuelve a conectarse. Sin embargo, ese desacoplamiento no es incondicional, ya que si la sesión no persiste más allá de la conexión, o si los mensajes se publican con QoS 0 que el servidor no está obligado a almacenar, un suscriptor desconectado nunca recibe los mensajes publicados, por lo que corresponde a la aplicación elegir la combinación adecuada para cada tipo de dato.

Complementando este comportamiento, MQTT suma dos mecanismos más. Un mensaje publicado con el indicador RETAIN se almacena en el servidor como último mensaje retenido de su tópico y se entrega a cada nuevo suscriptor, de modo que un cliente que se suscribe recibe de inmediato el estado más reciente aunque el publicador lo envíe de forma irregular. El mensaje de última voluntad (*Will Message*), por su parte, lo registra el cliente al conectarse y el servidor lo publica a los suscriptores si la conexión se cierra de forma anormal, por ejemplo cuando el cliente deja de comunicarse dentro del intervalo de keep-alive, que es el tiempo máximo que puede transcurrir entre dos paquetes enviados por el cliente. Esto permite notificar a los interesados de la desconexión de un dispositivo.

Por último, toda la comunicación en MQTT se realiza mediante paquetes de control, de los cuales la especificación define quince tipos. Entre ellos se encuentran CONNECT y CONNACK para establecer la conexión, PUBLISH para transportar los mensajes de aplicación, PUBACK y los demás paquetes de confirmación de los niveles de QoS, SUBSCRIBE y UNSUBSCRIBE para gestionar las suscripciones, y PINGREQ y PINGRESP para el keep-alive. Cada paquete se compone de un encabezado fijo, presente en todos ellos, seguido de un encabezado variable y de una carga útil que solo aparecen en algunos tipos. El primer byte del encabezado fijo indica el tipo de paquete y algunos indicadores, como el nivel de QoS y RETAIN en el caso de PUBLISH, y a continuación se codifica la longitud restante del paquete con un entero de longitud variable que ocupa un solo byte para valores de hasta 127. De este modo, el encabezado fijo ocupa apenas dos bytes en los paquetes pequeños, que es también el tamaño total de los paquetes de keep-alive. El uso de este tipo de encabezados resulta en una economización del costo de transporte, la cual se evalúa en el experimento de la Sección 5.2.

El contenido de los mensajes queda fuera del protocolo MQTT, que transporta la carga útil sin interpretarla. La versión 5.0 permite declarar su formato mediante las propiedades de indicador

BORRADOR

de formato y tipo de contenido, que el servidor reenvía sin modificar, pero no impone ninguno, aspecto que se retoma en la Sección 4. La representación de los datos queda así a cargo de la aplicación, que es el problema que aborda Protocol Buffers, descrito en la Sección 3.

3. Protocol Buffers

Una conexión TCP entrega una secuencia de bytes, y un broker MQTT reenvía el contenido de cada mensaje sin interpretarlo. Para que un dato producido por un microcontrolador sea leído correctamente por un servicio escrito en otro lenguaje y ejecutado en otra arquitectura, ambos extremos deben acordar cómo se traduce una estructura de datos a bytes y viceversa, incluyendo el orden de los bytes, el tamaño de los enteros y la forma de delimitar cada campo. En el modelo OSI esta tarea corresponde a la capa de presentación [4], pero la arquitectura TCP/IP no la incluye, por lo que cada aplicación debe resolverla por su cuenta [1].

Este problema no es nuevo. XDR (*External Data Representation*) y ASN.1 con sus reglas de codificación BER, esta última utilizada por SNMP, son soluciones clásicas que definen una representación de los datos independiente de la máquina [1]. Protocol Buffers puede entenderse como un sucesor moderno de esa misma idea.

Protocol Buffers, también conocido como Protobuf, es un mecanismo para serializar datos estructurados que es neutral respecto del lenguaje de programación y de la plataforma [6]. Fue desarrollado por Google, que lo publicó como software libre y mantiene implementaciones oficiales para una variedad de lenguajes [7]. Su propuesta es comparable a la de JSON, pero con mensajes más pequeños, un procesamiento más rápido y la generación de código nativo para cada lenguaje. Según su documentación, ofrece un formato de serialización para paquetes de datos estructurados y tipados de hasta unos pocos megabytes, adecuado tanto para el tráfico de red efímero como para el almacenamiento de datos a largo plazo, y que puede extenderse con nueva información sin invalidar los datos existentes ni obligar a actualizar el código [6]. Estas propiedades explican su amplia adopción por parte de grandes empresas de internet y en arquitecturas de microservicios [8].

3.1. Esquema y generación de código

La estructura de los datos se define una única vez en un archivo de esquema con extensión `.proto`, en el que cada mensaje se describe como un conjunto de campos con un tipo, un nombre y un número que lo identifica. Por ejemplo, la lectura de un sensor ambiental podría definirse así:

```
syntax = "proto3";

message Lectura {
  float temperatura = 1;
  float humedad     = 2;
  uint32 bateria    = 3;
}
```

A partir de este archivo, el compilador `protoc` genera el código fuente que permite construir, serializar y deserializar el mensaje en cada lenguaje de destino. De este modo, el `.proto` cumple a la vez la función de documentación y la de contrato explícito entre los equipos o servicios que intercambian los datos, y un mismo esquema produce implementaciones compatibles en todos los lenguajes soportados.

BORRADOR

3.2. Codificación binaria

En la codificación binaria de Protobuf, cada campo se transmite precedido de una etiqueta (*tag*) que combina el número de campo definido en el esquema con un tipo de codificación (*wire type*), que indica cómo leer el valor a continuación, como un entero de longitud variable, un valor de 32 o de 64 bits de longitud fija, o un bloque precedido de su longitud, que se usa para cadenas, bytes y mensajes anidados [9]. Se trata, en esencia, de una codificación de tipo-longitud-valor (TLV), como la de BER.

Los enteros se codifican como *varints*, en los que cada byte aporta siete bits del valor y utiliza el bit más significativo para indicar si el número continúa en el byte siguiente. Así, los valores pequeños, que son los más frecuentes, ocupan un solo byte. Por ejemplo, un campo número 1 con el valor 150 se codifica en tres bytes, `08 96 01`: el primero es la etiqueta y los dos restantes el valor [9]. Para los enteros con signo existe además la codificación *zigzag*, que intercala positivos y negativos para que los valores negativos pequeños también resulten cortos. El mensaje no incluye los nombres de los campos, que solo existen en el esquema, y en proto3 los campos que tienen su valor por defecto directamente no se transmiten [10]. Esta combinación es la que hace que un mensaje Protobuf pese varias veces menos que su equivalente en JSON y que se procese más rápido.

3.3. Compatibilidad entre versiones

Dado que los campos se identifican por número y no por posición ni por nombre, el esquema puede evolucionar sin romper a quienes usan versiones anteriores. Un programa que recibe un campo cuyo número no conoce simplemente lo ignora, de modo que los clientes viejos pueden leer mensajes nuevos (compatibilidad hacia adelante). En sentido inverso, un programa nuevo que lee un mensaje viejo encuentra los valores por defecto en los campos ausentes (compatibilidad hacia atrás). La condición para que esto funcione es no reutilizar números de campo, para lo cual el lenguaje ofrece la palabra clave `reserved`, que impide volver a asignar los números o nombres de campos eliminados [10]. Esta compatibilidad en ambos sentidos es la principal fortaleza de Protobuf frente a otros formatos binarios.

3.4. Limitaciones

Las mismas decisiones que reducen el tamaño de los mensajes tienen costos. En primer lugar, el formato no es legible por personas, por lo que la depuración requiere herramientas como `protoc --decode` o conversores a JSON. En segundo lugar, no es autodescriptivo: sin el esquema no es posible interpretar los datos, ya que el binario no indica qué mensaje contiene ni qué significa cada campo. En tercer lugar, la codificación no delimita los mensajes, por lo que el protocolo que los transporta debe indicar dónde termina cada uno. Por último, agrega al proyecto un paso de compilación y de generación de código, y resulta menos práctico que JSON para APIs públicas simples o para su uso directo desde navegadores, aunque para ese caso existen alternativas como gRPC-Web.

4. Stack MQTT y Protocol Buffers

5. Comparación con alternativas

5.1. MQTT, HTTP y CoAP

5.2. Protocol Buffers y JSON

BORRADOR

Desarrollo

1. MQTT sobre TCP y TLS

1.1. Conexión persistente y keep-alive

1.2. Cifrado con TLS

1.3. QoS MQTT sobre la confiabilidad de TCP

2. Capa de red

2.1. NAT y firewalls

2.2. Ruteo interno

2.3. Ruteo externo

2.4. Convergencia y conexiones de larga duración

3. Ruteo por tópicos y ruteo IP

4. Casos de uso

5. Diseño experimental

5.1. Plataforma común

5.2. Experimento 1: MQTT contra HTTPS para telemetría

5.2.1. Planteo

5.2.2. Implementación

5.2.3. Resultados

5.2.4. Análisis de resultados

5.3. Experimento 2: sincronización de osciladores acoplados

5.3.1. Planteo

5.3.2. Implementación

5.3.3. Resultados

5.3.4. Análisis de resultados

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] D. E. Comer, *Internetworking with TCP/IP, Volume One: Principles, Protocols, and Architecture*, 6.ª ed. Upper Saddle River, NJ: Pearson, 2014.

[2] J. F. Kurose y K. W. Ross, *Redes de computadoras. Un enfoque descendente*, 7.ª ed. Madrid: Pearson Educación, 2017.

[3] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[4] W. Stallings, *Comunicaciones y redes de computadores*, 7.ª ed. Madrid: Pearson Educación, 2004.

[5] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[6] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[7] protocolbuffers, «protobuf: Protocol Buffers - Google's data interchange format». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://github.com/protocolbuffers/protobuf

[8] MQTTX Team, «How to Publish and Receive Protobuf Messages within MQTT?». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://www.emqx.com/en/blog/how-to-publish-and-receive-protobuf-messages-within-mqtt

[9] Google, «Encoding». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/encoding/

[10] Google, «Language Guide (proto 3)». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/proto3/

[11] L. Zhang, «Building Facebook Messenger». [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[12] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[13] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[14] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[15] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

BORRADOR